HW5: Individual Assignment
Manuel Agraz Vallejo

I built a total of 2 nodes for the navigation component of this assignment: a **go_to_goal** and a **visit_rooms** node. The **go_to_goal** node creates an action client to the **/navigate_to_pose** action server. It receives (x,y) coordinates of a goal and sends them to the action server. It then logs the current position progress using the feedback callback function. The second node is the **visit_rooms** node. This node has a hardcoded set of waypoints, each representing a room. It then creates two action clients, one for the **/navigate_to_pose** action server, which takes care of reaching the room waypoint and another for the **/spin** action server, which takes care of spinning inside a room once the waypoint is reached. The **visit_rooms** node keeps track of the visited rooms and selects the next room waypoint in a fixed order. Additionally, the node publishes a MarkerArray representing the waypoints that will be visited.

The biggest challenge of using Nav2 was figuring out the right parameters to tweak for the navigation; the biggest ones were the inflation radius and the robot radius. These two parameters affect how the costmap for navigation is created; thus, getting them just right was the difference between the robot finding paths or giving up after not finding any path.

For mapping the house environment, I used the slam_toolbox package on its own, without the Nav2 tooling. I stumbled upon some issues trying to get the slam_toolbox and Nav2 working together, and found out that the slam_toolbox alone is enough to do SLAM. After running the slam_toolbox, I ran a teleoperation node for the simulated turtlebot and mapped the house. This part was challenging since my map kept getting distorted at certain spots in the house, so I had to keep creating checkpoints for the map in case of distortion. Once I figured out the checkpoint system by storing the SLAM pose graphs, I was able to map the entire house.

This assignment helped me get familiar with using the Nav2 and the slam_toolbox. I was surprised by how stable the Nav2 stack is once the parameters are tuned to the robot in use. It only gets lost if you purposefully set its initial pose in a faraway pose, and it can’t navigate if the robot parameters are not set properly (robot_radius and inflation_radius for example).